
Software Requirements Specification

For

Integrated Platform for Crowdsourced Ocean Hazard Reporting and Social Media Analytics

Version 1.0 approved

Prepared by

B.Poornima 1602-23-737-032

K.SaiRevanth 1602-23-737-042

Vaibhav Shahi 1602-23-737-060

Vasavi College of Engineering

04-04-2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction	1
1.1 Purpose.....	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	2
1.4 Product Scope	2
1.5 References	3
2. Overall Description	3
2.1 Product Perspective	3
2.2 Product Functions.....	4
2.3 User Classes and Characteristics	4
2.4 Operating Environment	5
2.5 Design and Implementation Constraints.....	6
2.6 User Documentation.....	6
2.7 Assumptions and Dependencies	7
3. External Interface Requirements	7
3.1 User Interfaces.....	7-8
3.2 Hardware Interfaces	8
3.3 Software Interfaces.....	9
3.4 Communications Interfaces	10
4. System Features	11
4.1 System Feature 1	11
4.2 System Feature 2	12
4.3 System Feature 3	13
4.4 System Feature 4	14
5. Other Nonfunctional Requirements	15
5.1 Performance Requirements	15
5.2 Safety Requirements.....	15
5.3 Security Requirements	16
5.4 Software Quality Attributes.....	17
5.5 Business Rules	17
6. Other Requirements	18
Appendix A: Glossary	19
Appendix B: Analysis Models	19
Appendix C: To Be Determined List	28

Revision History

Name	Date	Reason For Changes	Version
Initial Implementation	04-04-2026	Additional Features should be added	1
Final Implementation	08-05-2026	-	2

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of the system titled “**Integrated Platform for Crowdsourced Ocean Hazard Reporting and Social Media Analytics.**”

The purpose of this document is to provide a detailed description of the system’s features, behavior, and constraints to guide developers, stakeholders, and evaluators in understanding the system design and implementation.

The scope of this system includes a **mobile and web-based platform** that enables citizens to report ocean hazards in real time and allows authorities to monitor, analyze, and respond to such events using geospatial visualization and AI-driven social media analytics.

This SRS covers the overall system, including:

- User interfaces (mobile and web)
- Backend services and APIs
- Database and geospatial components
- AI/NLP modules for social media analysis

1.2 Document Conventions

This document follows standard Software Requirements Specification (SRS) conventions to ensure clarity and consistency.

- All requirements are written in a clear and structured manner using simple and precise language.
- Functional requirements are listed using numbered bullet points for easy reference.
- Keywords such as “**shall**” indicate mandatory requirements, while “**should**” indicate recommended features.
- Section headings and subheadings are formatted consistently to maintain readability.
- Important terms and system components are highlighted using bold formatting.
- Each requirement is considered independent and may be assigned its own priority level (High, Medium, Low) based on its importance.
- Diagrams and tables are used where necessary to improve understanding of system architecture and workflows.

1.3 Intended Audience and Reading Suggestions

This document is intended for various stakeholders involved in the development and use of the system:

- **Developers:** To understand system requirements, architecture, and implementation details.
- **Project Managers:** To plan, monitor, and manage project progress and resources.
- **Testers:** To design test cases based on functional and non-functional requirements.
- **End Users (Citizens, Officials, Analysts):** To understand system functionalities and usage.
- **Documentation Writers:** To prepare user manuals and technical guides.

This SRS is organized into sections covering the system overview, functional requirements, non-functional requirements, system architecture, and constraints.

For effective understanding:

- Readers should begin with the **Introduction** section for an overview of the system.
- Developers and testers should focus on the **Functional and Non-Functional Requirements** sections.
- Project managers should review the **Overall Description and Constraints** sections.
- End users may refer to the **System Features and User Interface descriptions**.

This structured approach ensures that each stakeholder can easily access the information relevant to their role.

1.4 Product Scope

The proposed system, “**Integrated Platform for Crowdsourced Ocean Hazard Reporting and Social Media Analytics**,” is designed to enhance disaster management and coastal safety by integrating real-time citizen reporting with AI-driven social media analysis.

The primary purpose of this software is to provide a unified platform where users can report ocean hazards such as high waves, coastal flooding, and abnormal sea conditions, while simultaneously enabling authorities to monitor and analyze hazard-related information from multiple sources. The system improves situational awareness through geospatial visualization, dynamic hotspot detection, and real-time data aggregation.

The key objectives of the system include:

- Enabling real-time, geotagged hazard reporting by citizens
- Leveraging AI and NLP to extract insights from social media
- Supporting faster and more informed decision-making for disaster response
- Enhancing coordination between citizens and disaster management authorities

This software aligns with broader goals of **disaster risk reduction, public safety, and sustainable coastal management**, supporting initiatives related to climate resilience and smart disaster response systems.

1.5 References

“Indian National Centre for Ocean Information Services.” *INCOIS*, Ministry of Earth Sciences, <https://incois.gov.in>.

“PostgreSQL Documentation.” *PostgreSQL Global Development Group*, <https://www.postgresql.org/docs/>.

“PostGIS Documentation.” *PostGIS Project*, <https://postgis.net/documentation/>.

“FastAPI Documentation.” *FastAPI*, <https://fastapi.tiangolo.com/>.

“React Documentation.” *React*, Meta Platforms, <https://react.dev/>.

“Flutter Documentation.” *Flutter*, Google, <https://docs.flutter.dev/>.

“Twitter Developer Platform.” *X (Twitter)*, <https://developer.twitter.com/>.

“YouTube Data API.” *Google Developers*, <https://developers.google.com/youtube/v3>.

2. Overall Description

2.1 Product Perspective

The proposed system, “**Integrated Platform for Crowdsourced Ocean Hazard Reporting and Social Media Analytics**,” is a new, standalone application designed to enhance existing ocean hazard monitoring and disaster management systems.

Currently, organizations like the Indian National Centre for Ocean Information Services (INCOIS) rely on satellite data, ocean buoys, and predictive models to issue early warnings. However, these systems lack real-time integration of ground-level observations from citizens and unstructured data from social media platforms.

This product is designed to complement and extend existing systems by acting as an additional intelligence layer. It integrates crowdsourced reporting, geospatial data processing, and AI-driven social media analytics into a unified platform.

The system consists of the following major components:

- **User Layer:** Citizens, officials, and analysts interacting through mobile and web interfaces
- **Frontend Layer:** Mobile application (Flutter) and web dashboard (React)
- **Backend Layer:** API services (FastAPI) handling business logic and data processing
- **Database Layer:** PostgreSQL with PostGIS for geospatial data storage

- **AI/NLP Module:** Processes social media data and extracts hazard-related insights
- **External Interfaces:** Integration with social media platforms (Twitter, YouTube) and optional connection to early warning systems

This system interacts with external data sources and existing disaster management frameworks while maintaining its own independent functionality.

2.2 Product Functions

The system provides the following major functions:

- **User Registration and Authentication:**
Enables citizens, officials, and analysts to securely register and access the system based on their roles.
- **Crowdsourced Hazard Reporting:**
Allows users to submit geotagged reports of ocean hazards along with images or videos.
- **Real-Time Data Aggregation:**
Collects and processes data from user reports and external sources in real time.
- **Geospatial Visualization:**
Displays hazard reports on an interactive map with dynamic hotspot detection.
- **Social Media Monitoring and Analysis:**
Extracts and analyzes hazard-related information from social media using NLP techniques.
- **Alert and Notification System:**
Generates alerts based on detected hazards and notifies relevant authorities.
- **Role-Based Dashboard:**
Provides customized dashboards for citizens, officials, and analysts with relevant data and insights.
- **Offline Data Collection and Sync:**
Allows users to record reports without internet and sync them when connectivity is restored.

2.3 User Classes and Characteristics

The system is designed to support multiple user classes based on roles, access levels, and usage patterns:

2.3.1 1. Citizens (Primary Users)

- Coastal residents, volunteers, and general public
- Use the mobile/web application to report ocean hazards
- Can upload geotagged images, videos, and descriptions
- Require simple, user-friendly interfaces
- Moderate technical knowledge
- Most important user class as they provide real-time ground data

2.3.2 Government Officials (Primary Users)

- Disaster management authorities and coastal monitoring agencies
- Access dashboards to monitor hazard reports and hotspots
- Use system for decision-making and response coordination
- Require secure access and reliable real-time data
- Moderate to high technical understanding

2.3.3 Analysts (Secondary Users)

- Data analysts and researchers
- Analyze trends, patterns, and social media insights
- Use advanced features like filtering, visualization, and reports
- Require higher technical expertise

2.3.4 System Administrators (Support Users)

- Manage system operations, user roles, and data integrity
- Handle maintenance, updates, and security controls
- High technical expertise required

2.3.5 User Priority Classification

- **Primary Users:** Citizens, Government Officials
- **Secondary Users:** Analysts
- **Support Users:** System Administrators

2.4 Operating Environment

The system is designed to operate in a distributed environment supporting both mobile and web platforms.

- **Hardware Platform:**
The system can run on standard smartphones (Android/iOS), personal computers, and cloud-based servers.
- **Operating Systems:**
Mobile devices: Android and iOS
Web access: Windows, Linux, or macOS through modern web browsers
- **Frontend Environment:**
Mobile application developed using Flutter
Web dashboard accessible via browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge
- **Backend Environment:**
Server-side application built using FastAPI (Python) running on Linux/Windows servers
- **Database Environment:**
PostgreSQL with PostGIS extension for geospatial data handling
- **Supporting Software and Tools:**
RabbitMQ for message queuing

- Docker (optional) for containerized deployment
- Cloud platforms (AWS/Azure/GCP) for hosting and storage
- **External Interfaces:**
Integration with social media platforms (Twitter, YouTube) and external APIs for data collection

The system is designed to be scalable, ensuring smooth operation across different environments and varying user loads.

2.5 Design and Implementation Constraints

The development of the system is subject to the following constraints:

- **Technology Constraints:**
The system is developed using specific technologies such as FastAPI (Python), React.js, Flutter, and PostgreSQL with PostGIS, limiting flexibility in choosing alternative frameworks.
- **Hardware Constraints:**
The application must function efficiently on standard smartphones and computers with limited processing power, memory, and network connectivity.
- **Network Constraints:**
The system depends on internet connectivity for real-time data transmission, although offline data collection and later synchronization must be supported.
- **Integration Constraints:**
The system must integrate with external platforms such as social media APIs (Twitter, YouTube), which may have access limitations, rate limits, or policy restrictions.
- **Security Constraints:**
The system must ensure secure data handling, including user authentication, data privacy, and protection against unauthorized access.
- **Regulatory and Policy Constraints:**
The system must comply with data privacy laws and government regulations related to disaster management and public data usage.
- **Scalability Constraints:**
The system must handle increasing user load and large volumes of real-time data without performance degradation.
- **Design and Coding Standards:**
Developers must follow standard programming practices, modular design, and proper documentation to ensure maintainability and future scalability.

2.6 User Documentation

The system will include the following user documentation components to assist different types of users:

- **User Manual:**
A comprehensive guide explaining system features, user roles, and step-by-step instructions for using the mobile and web applications.

- **Quick Start Guide:**
A short document to help new users quickly understand how to register, report hazards, and navigate the system.
- **Online Help / Help Section:**
Built-in help within the application providing guidance, FAQs, and troubleshooting support.
- **Tutorials and Demo Videos:**
Visual guides demonstrating how to use key features such as reporting hazards, viewing dashboards, and interpreting maps.
- **Technical Documentation:**
Documentation for developers and administrators covering system architecture, APIs, and deployment procedures.
- **API Documentation:**
Auto-generated API documentation (e.g., FastAPI Swagger UI) for backend services.

2.7 Assumptions and Dependencies

The following assumptions and dependencies are considered during the development of the system:

2.7.1 Assumptions

- Users (citizens, officials, analysts) have access to smartphones or computers with basic internet connectivity.
- Users are willing to actively report hazard-related events and share accurate information.
- Social media platforms provide accessible data through APIs or scraping mechanisms.
- The system will be deployed on reliable cloud infrastructure with sufficient scalability.
- External APIs (e.g., for social media and weather data) will remain available and stable.
- Users have basic technical knowledge to operate mobile and web applications.

2.7.2 Dependencies

- The system depends on **third-party services** such as social media APIs (Twitter, YouTube) for data collection.
- It relies on **external libraries and frameworks** including FastAPI, React, Flutter, and NLP models.
- The system depends on **database systems** like PostgreSQL with PostGIS for geospatial data processing.
- Integration with **cloud services** (AWS/Azure/GCP) is required for storage and deployment.
- The accuracy of AI/NLP modules depends on the quality and availability of training data.
- The system may depend on existing **early warning systems** for additional data inputs.

3. External Interface Requirements

3.1 User Interfaces

The system provides user interfaces for different user roles, including citizens, government officials, analysts, and administrators. The interfaces are designed to be intuitive, responsive, and accessible across mobile and web platforms.

- **Mobile Application Interface (Flutter):**
Provides a user-friendly interface for citizens to register, log in, and submit geotagged hazard reports with images or videos. The interface includes simple navigation, location selection, and media upload features.
- **Web Dashboard Interface (React):**
Used by officials and analysts to monitor real-time data through interactive maps, charts, and dashboards. It supports filtering by location, event type, and time, along with hotspot visualization.
- **User Interface Characteristics:**
 - Consistent layout and navigation across all screens
 - Use of standard UI elements such as buttons (Submit, Login, Logout, Help)
 - Responsive design compatible with different screen sizes
 - Clear error messages and input validation for user actions
 - Role-based interface showing relevant features for each user
- **GUI Standards and Design Guidelines:**
 - Clean and minimal design with ocean-themed color palette (blue/teal tones)
 - Use of icons for actions such as reporting, alerts, and maps
 - Accessibility considerations such as readable fonts and contrast
- **Error Handling and Feedback:**
 - Display of meaningful error messages for invalid inputs or failed operations
 - Confirmation messages for successful actions (e.g., report submission)
- **Help and Support Features:**
 - In-app help section and FAQs
 - Tooltips and guidance for new users

The detailed design of the user interface, including screen layouts and workflows, is documented separately in the User Interface Specification.

3.2 Hardware Interfaces

The system interacts with various hardware components through standard interfaces to support data collection, processing, and user interaction.

- **Supported Devices:**
The system is designed to operate on smartphones (Android/iOS), personal computers, and cloud-based servers.
- **Mobile Device Sensors:**
The mobile application utilizes device hardware such as:
 - **GPS Sensor:** For capturing geolocation of hazard reports
 - **Camera:** For capturing images and videos of ocean hazards
 - **Network Interface:** For transmitting data to the backend server
- **Server Hardware:**
The backend system runs on cloud or local servers that handle data processing, storage, and API requests. These servers support high-performance computing and scalable storage.
- **User Interaction Devices:**
 - Touchscreen interfaces for mobile devices

- Keyboard and mouse for web-based dashboards
- **Data and Control Interaction:**
The software collects data from hardware components (GPS, camera), processes it, and transmits it to the backend through internet connectivity.
- **Communication Protocols:**
 - HTTP/HTTPS for data transmission between client and server
 - REST APIs for interaction between frontend and backend

The system is designed to be compatible with commonly available hardware devices, ensuring ease of use and wide accessibility.

3.3 Software Interfaces

The system interacts with various software components, frameworks, and external services to enable data processing, storage, and communication.

3.3.1 Database Interface

- **Software:** PostgreSQL (v14+) with PostGIS extension
- **Purpose:** Stores user data, hazard reports, geospatial information, and system logs
- **Data Exchange:**
 - Input: User details, hazard reports, location data
 - Output: Retrieved data for dashboard visualization and analysis

3.3.2 Backend Framework Interface

- **Software:** FastAPI (Python)
- **Purpose:** Handles business logic, API requests, authentication, and data processing
- **Communication:** RESTful APIs using HTTP/HTTPS
- **Data Exchange:**
 - Input: Requests from frontend and external APIs
 - Output: JSON responses to frontend and services

3.3.3 Frontend Interface

- **Software:** React.js (Web), Flutter (Mobile)
- **Purpose:** Provides user interface for interaction with the system
- **Communication:** REST API calls to backend
- **Data Exchange:**
 - Input: User inputs (reports, filters, login data)
 - Output: Dashboard data, maps, alerts

3.3.4 Social Media Interfaces

- **Software:** Twitter API, YouTube Data API
- **Purpose:** Fetch hazard-related posts and comments for analysis
- **Data Exchange:**
 - Input: Social media content (text, metadata)

- Output: Processed insights (keywords, sentiment, trends)

3.3.5 AI/NLP Module Interface

- **Software:** NLP libraries / models (e.g., Transformers, Gemini API)
- **Purpose:** Classifies and analyzes text and images
- **Data Exchange:**
 - Input: Social media text, images
 - Output: Classified hazard data, sentiment scores

3.3.6 Message Queue Interface

- **Software:** RabbitMQ
- **Purpose:** Handles asynchronous processing tasks
- **Communication:** Message-based communication between backend and workers

3.3.7 Operating System Interface

- **Software:** Windows, Linux, macOS
- **Purpose:** Provides runtime environment for backend and frontend applications

3.3.8 Data Sharing and Communication

- Data is exchanged between components using **JSON format over REST APIs**
- Secure communication is ensured using **HTTPS protocols**
- Data is shared across components through centralized database and API services

3.4 Communications Interfaces

The system requires reliable communication interfaces to enable data exchange between users, servers, and external services.

- **Network Communication:**
The system uses **HTTP/HTTPS protocols** for communication between frontend (mobile/web) and backend services through RESTful APIs.
- **Web Browser Interface:**
Users access the web dashboard via modern browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge over the internet.
- **Mobile Communication:**
The mobile application communicates with backend servers using secure internet connections (HTTPS) for submitting reports and retrieving data.
- **Data Format:**
All data exchanged between system components is formatted in **JSON** for consistency and ease of processing.
- **External Communication:**
Integration with external services such as social media platforms (Twitter, YouTube) is performed through APIs using standard web protocols.

- **Message Handling:**
Asynchronous communication between system components is handled using message queues (e.g., RabbitMQ) to ensure efficient data processing.
- **Security and Encryption:**
 - Secure communication using **HTTPS (SSL/TLS encryption)**
 - Authentication mechanisms (JWT tokens) for authorized access
 - Protection against unauthorized data access
- **Data Transfer and Synchronization:**
 - Supports real-time data updates with minimal latency
 - Offline data is synchronized automatically when connectivity is restored

The system is designed to ensure secure, efficient, and reliable communication across all components.

4. System Features

4.1 CrowdSourced Hazard Reporting

4.1.1 Description and Priority

This feature enables users (citizens) to report ocean hazards such as high waves, coastal flooding, and abnormal sea conditions by submitting geotagged data along with images or videos through the mobile or web application.

Priority: High

4.1.2 Stimulus / Response Sequences

Stimulus 1: User selects “Report Hazard” option

→ **Response:** System opens report submission form

Stimulus 2: User enters details (location, description, media) and submits

→ **Response:** System validates input and uploads data to server

Stimulus 3: Valid data submitted

→ **Response:** System stores report in database and displays confirmation message

Stimulus 4: Invalid or incomplete input

→ **Response:** System displays error message and prompts user to correct input

4.1.3 Functional Requirements

REQ-1: The system shall allow users to create and submit hazard reports.

REQ-2: The system shall capture geolocation data using GPS or manual input.

REQ-3: The system shall allow users to upload images and videos as part of the report.

REQ-4: The system shall validate all required input fields before submission.

REQ-5: The system shall store submitted reports in the database.

REQ-6: The system shall display a confirmation message after successful submission.

REQ-7: The system shall display appropriate error messages for invalid or missing inputs.

REQ-8: The system shall support offline report submission and sync data when connectivity is restored.

4.2 GeoSpatial Visualization and Hotspot Detection

4.2.1 Description and Priority

This feature enables the system to display crowdsourced hazard reports on an interactive map and dynamically generate hotspots based on report density and severity. It helps officials and analysts visualize affected regions in real time.

Priority: High

4.2.2 Stimulus / Response Sequences

Stimulus 1: User opens the dashboard

→ **Response:** System loads map with current hazard reports

Stimulus 2: New report is submitted

→ **Response:** System updates map and recalculates hotspot regions

Stimulus 3: User applies filters (location, time, event type)

→ **Response:** System updates map view based on selected filters

Stimulus 4: No data available for selected filters

→ **Response:** System displays “No data available” message

4.2.3 Functional Requirements

REQ-1: The system shall display hazard reports on an interactive geospatial map.

REQ-2: The system shall update the map in real time when new reports are added.

REQ-3: The system shall generate hotspots based on report density.

REQ-4: The system shall allow users to filter data by location, event type, and time.

REQ-5: The system shall visually differentiate hotspots using color gradients or markers.

REQ-6: The system shall provide detailed information when a user selects a report on the map.

REQ-7: The system shall handle cases where no data is available and display appropriate messages.

4.3 Social Media Monitoring and Analysis

4.3.1 Description and Priority

This feature enables the system to collect, process, and analyze hazard-related data from social media platforms such as Twitter and YouTube using Natural Language Processing (NLP). It helps in identifying trends, public sentiment, and potential hazard indicators.

Priority: High

4.3.2 Stimulus / Response Sequences

Stimulus 1: System fetches social media data periodically

→ **Response:** Data is processed and filtered for hazard-related content

Stimulus 2: Relevant keywords or patterns detected

→ **Response:** System classifies content and updates analytics dashboard

Stimulus 3: User views social media insights

→ **Response:** System displays trends, keywords, and sentiment analysis

Stimulus 4: No relevant data found

→ **Response:** System displays appropriate message

4.3.3 Functional Requirements

REQ-1: The system shall collect data from social media platforms.

REQ-2: The system shall filter and identify hazard-related content using NLP techniques.

REQ-3: The system shall classify posts based on type of hazard.

REQ-4: The system shall analyze sentiment (positive, negative, neutral).

REQ-5: The system shall display trends and keyword frequency.

REQ-6: The system shall update analytics in near real time.

REQ-7: The system shall handle cases where no relevant data is found.

4.4 Alert and Notification System

4.4.1 Description and Priority

This feature enables the system to generate alerts based on hazard reports and social media analysis, and notify relevant authorities for quick response.

Priority: High

4.4.2 Stimulus / Response Sequences

Stimulus 1: System detects high report density or critical condition
→ **Response:** System generates alert

Stimulus 2: Alert triggered
→ **Response:** Notifications sent to officials

Stimulus 3: User views alert dashboard
→ **Response:** System displays alert details

4.4.3 Functional Requirements

REQ-1: The system shall generate alerts based on predefined thresholds.

REQ-2: The system shall notify officials in real time.

REQ-3: The system shall display alerts on the dashboard.

REQ-4: The system shall allow users to view alert details.

REQ-5: The system shall support multiple alert levels (low, medium, high).

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The system shall meet the following performance requirements to ensure efficient and reliable operation:

- **Response Time:**
The system shall respond to user requests (e.g., login, report submission, dashboard loading) within **2–5 seconds** under normal network conditions.
- **Real-Time Data Processing:**
Hazard reports and social media data shall be processed and reflected on the dashboard within **5–10 seconds** of submission or detection.
- **Concurrent Users:**
The system shall support at least **500–1000 concurrent users** without significant performance degradation.
- **Data Throughput:**
The system shall efficiently handle large volumes of incoming data, including multiple hazard reports and social media inputs per second.
- **Map and Visualization Performance:**
The geospatial dashboard shall load and render map data within **3–5 seconds**, even with high data density.
- **Alert Generation Time:**
The system shall generate and deliver alerts within **5 seconds** of detecting a critical condition.
- **Offline Synchronization:**
Data collected offline shall be synchronized with the server within **10–15 seconds** after network connectivity is restored.
- **System Availability:**
The system shall maintain at least **99% uptime**, ensuring availability during critical disaster situations.

5.2 Safety Requirements

The system shall ensure safe operation to prevent any potential harm, misinformation, or misuse during disaster situations:

- **Accurate Information Handling:**
The system shall ensure that hazard data is validated to minimize the risk of false or misleading information.
- **Data Integrity:**
The system shall protect data from loss, corruption, or unauthorized modification during transmission and storage.
- **Controlled Alert Generation:**
The system shall generate alerts based on verified data and predefined thresholds to avoid false alarms or panic.

- **User Input Validation:**
The system shall validate all user inputs to prevent incorrect or harmful data submissions.
- **System Reliability:**
The system shall maintain consistent operation during critical events to avoid downtime that could impact safety.
- **Access Control:**
The system shall restrict access to sensitive data and functionalities to authorized users only.
- **Compliance with Regulations:**
The system shall adhere to applicable government policies and disaster management guidelines related to public safety and data usage.
- **Error Handling:**
The system shall handle failures gracefully and provide appropriate messages without exposing critical system details.

5.3 Security Requirements

The system shall ensure secure access, data protection, and privacy for all users and system components:

- **User Authentication:**
The system shall provide secure login mechanisms using username/password and role-based authentication.
- **Authorization and Access Control:**
The system shall enforce role-based access control to restrict functionalities based on user roles (citizen, official, analyst, admin).
- **Data Encryption:**
All data transmitted between client and server shall be encrypted using **HTTPS (SSL/TLS)** protocols.
- **Data Privacy:**
The system shall protect personal user data and ensure it is not disclosed to unauthorized entities.
- **Secure Storage:**
Sensitive data such as user credentials shall be securely stored using encryption or hashing techniques.
- **Protection Against Attacks:**
The system shall implement safeguards against common security threats such as SQL injection, cross-site scripting (XSS), and unauthorized access.
- **Session Management:**
The system shall maintain secure user sessions and automatically log out inactive users after a defined period.
- **API Security:**
All APIs shall require authentication tokens (e.g., JWT) to ensure secure communication between components.
- **Compliance with Regulations:**
The system shall comply with applicable data protection and cybersecurity regulations.

5.4 Software Quality Attributes

The system shall satisfy the following quality attributes to ensure efficiency, usability, and reliability:

- **Usability:**
The system shall provide a simple and intuitive user interface, enabling users to perform key actions (e.g., report submission) within **3–4 steps**.
- **Reliability:**
The system shall operate consistently with minimal failures and maintain at least **99% uptime** during normal operations.
- **Availability:**
The system shall be accessible 24/7, especially during critical disaster situations.
- **Performance Efficiency:**
The system shall process and display data within defined response times (2–5 seconds for standard operations).
- **Scalability:**
The system shall handle increasing numbers of users and data without significant performance degradation.
- **Maintainability:**
The system shall be designed with modular architecture to allow easy updates, debugging, and enhancements.
- **Portability:**
The system shall be compatible across multiple platforms, including Android, iOS, and web browsers.
- **Security:**
The system shall ensure secure handling of data and user access through authentication and encryption mechanisms.
- **Interoperability:**
The system shall integrate with external services such as social media APIs and geospatial tools.
- **Robustness:**
The system shall handle invalid inputs, network failures, and unexpected conditions without crashing.
- **Testability:**
The system shall support unit testing and integration testing to verify functionality and performance.

5.5 Business Rules

The system follows the below operating principles based on user roles and system behavior:

- **Role-Based Access:**
Only authorized users can access specific functionalities. Citizens can submit reports, while officials and analysts can view and analyze data.
- **Report Submission Rules:**
Only registered users are allowed to submit hazard reports. Each report must include valid location data and relevant details.
- **Data Visibility:**
Citizens can view general hazard information, while officials and analysts have access to detailed dashboards and analytics.

- **Alert Handling:**
Alerts are generated automatically based on predefined conditions and are primarily visible to authorized officials.
- **System Usage Conditions:**
The system shall operate continuously and support real-time data updates during active hazard events.
- **Data Validation Principle:**
All incoming data (user reports and social media inputs) shall be validated before being used for analysis or alert generation.
- **Security Principle:**
Users shall only access data and features permitted by their roles, ensuring data privacy and system integrity.
- **Offline Operation:**
Users can submit reports offline, which will be synchronized once network connectivity is restored.

6. Other Requirements

The following additional requirements are considered for the system:

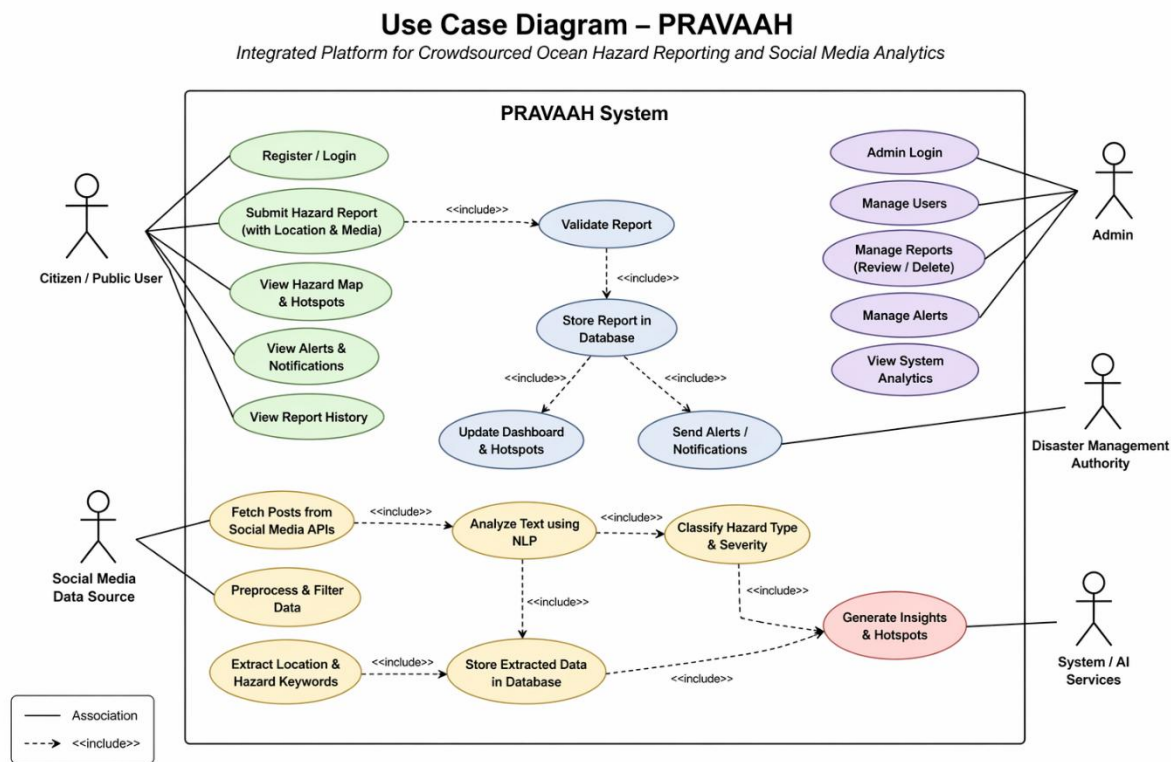
- **Database Requirements:**
The system shall use PostgreSQL with PostGIS to support efficient storage and processing of geospatial data. The database shall ensure data consistency, integrity, and backup mechanisms.
- **Internationalization Requirements:**
The system shall support multiple languages to ensure accessibility for users from different regions, especially coastal communities.
- **Legal and Regulatory Requirements:**
The system shall comply with applicable data protection laws and government regulations related to disaster management and public data usage.
- **Data Backup and Recovery:**
The system shall implement regular data backup and recovery mechanisms to prevent data loss during failures.
- **Logging and Monitoring:**
The system shall maintain logs for user activities, system events, and errors to support monitoring and debugging.
- **Reuse and Scalability:**
The system shall be designed with reusable components and scalable architecture to support future enhancements and integration with other disaster management systems.
- **Compatibility Requirements:**
The system shall be compatible with modern devices, browsers, and operating systems.

Appendix A: Glossary

- **INCOIS:** Indian National Centre for Ocean Information Services
- **NLP:** Natural Language Processing – Technique used to analyze and understand text data
- **API:** Application Programming Interface – Enables communication between software components
- **GPS:** Global Positioning System – Used to capture location data
- **UI:** User Interface – The visual interaction layer of the system
- **DB:** Database – Structured storage of system data
- **PostGIS:** Spatial extension of PostgreSQL used for geospatial data handling
- **REST API:** Representational State Transfer – Web service communication standard
- **HTTP/HTTPS:** HyperText Transfer Protocol (Secure) – Communication protocol for data transfer
- **JWT:** JSON Web Token – Used for secure authentication
- **Dashboard:** Visual interface displaying data analytics and system insights
- **Hotspot:** Area with high concentration of hazard reports or activity
- **Crowdsourcing:** Collecting data or information from a large group of users
- **Geospatial Data:** Data associated with geographic locations
- **Hazard Event:** Any ocean-related abnormal condition such as high waves, flooding, or tsunami

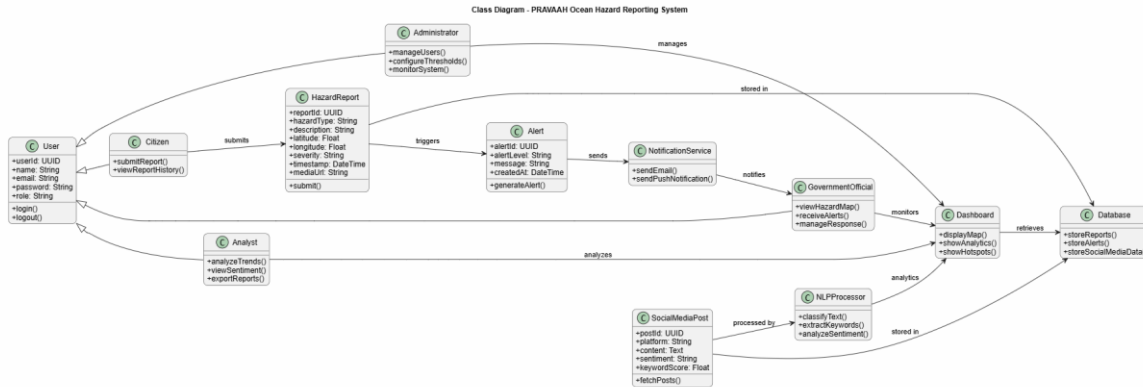
Appendix B: Analysis Models

1. UseCase Diagram:

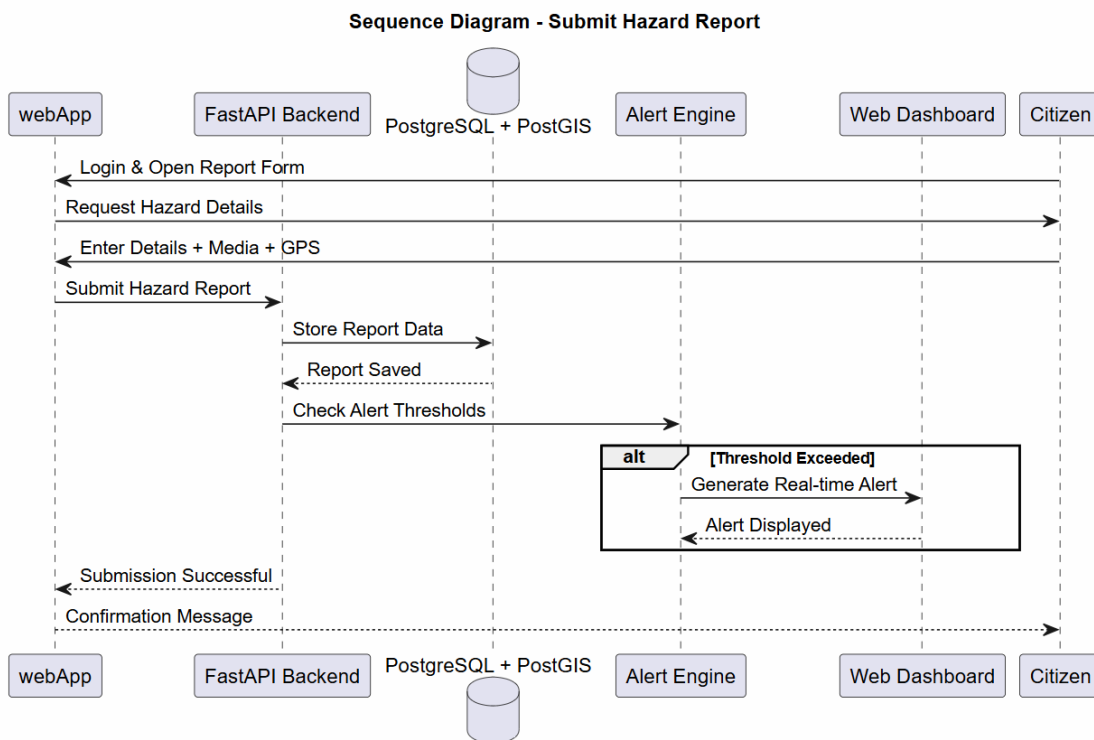


Software Requirements Specification for Integrated Platform for Crowdsourced Ocean Hazard Reporting and Social Media Analytics

2. Class Diagram:

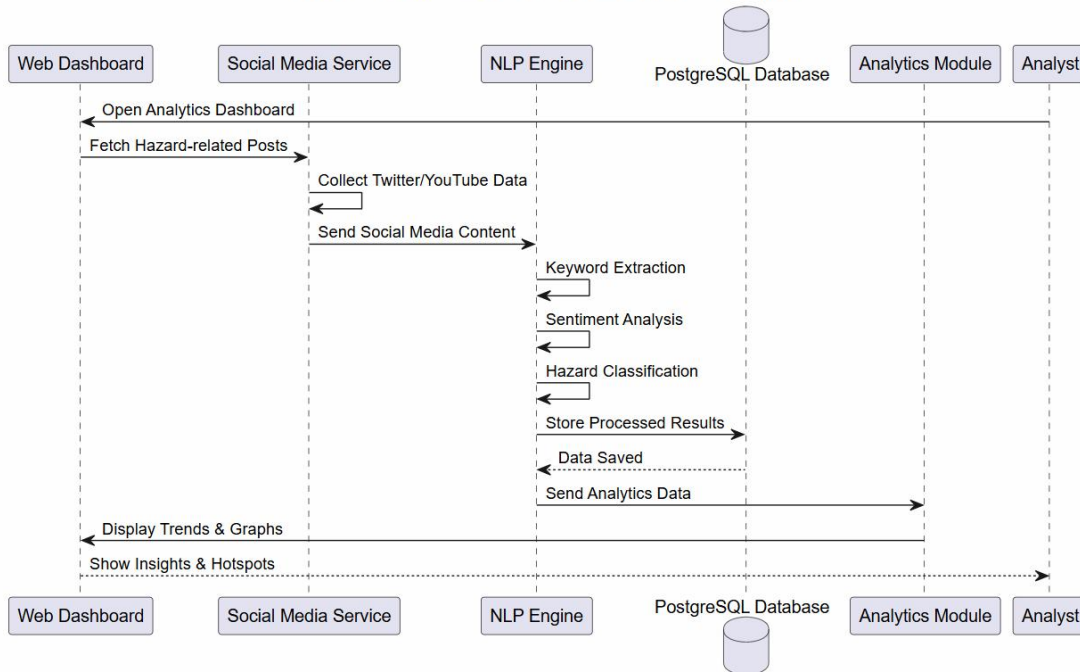


3. Sequence Diagrams:

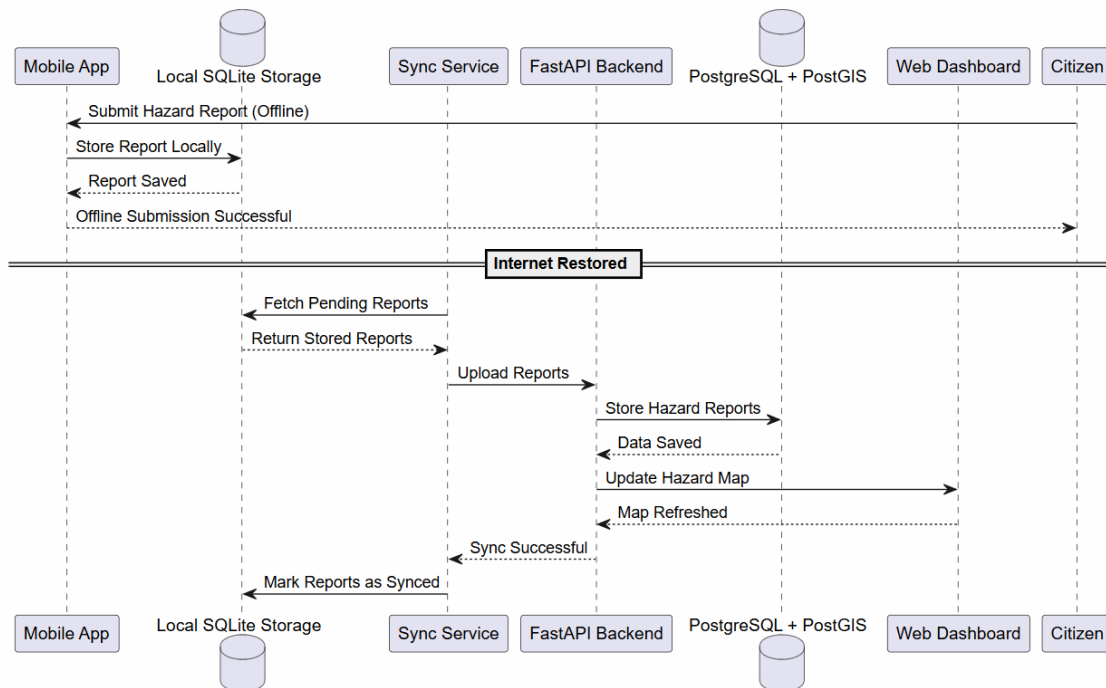


Software Requirements Specification for Integrated Platform for Crowdsourced Ocean Hazard Reporting and Social Media Analytics

Sequence Diagram - Social Media Monitoring & NLP Analysis

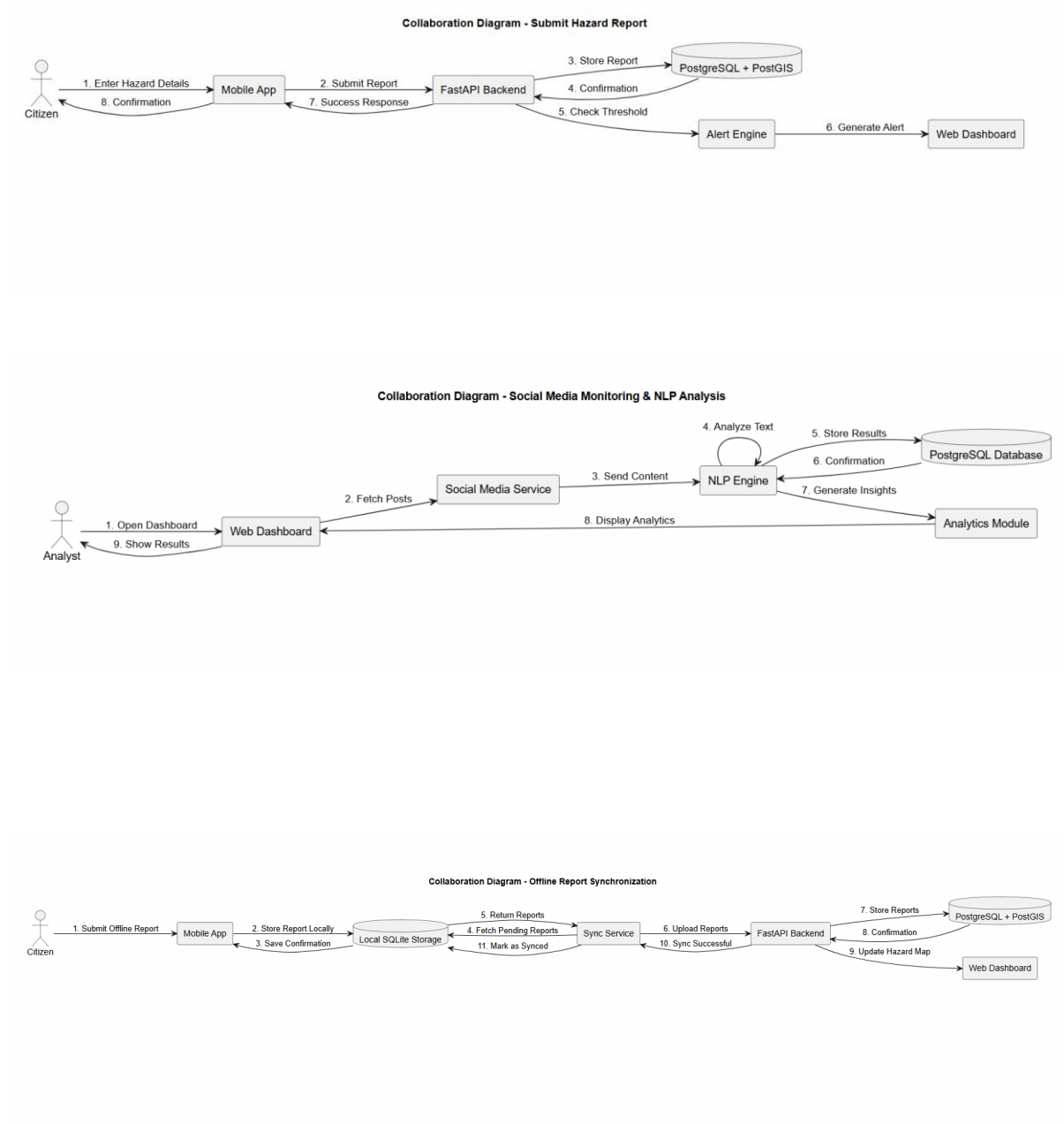


Sequence Diagram - Offline Report Synchronization



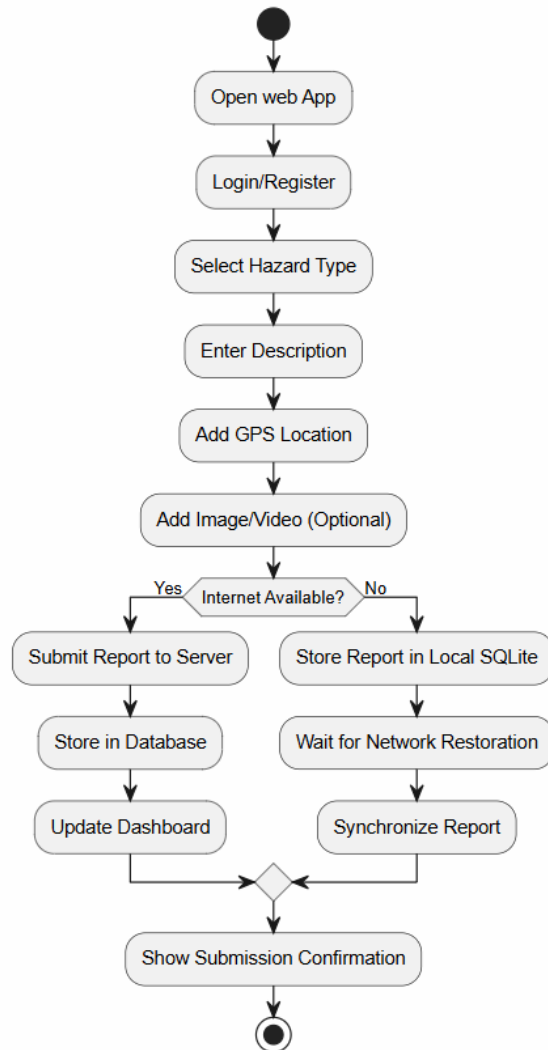
Software Requirements Specification for Integrated Platform for Crowdsourced Ocean Hazard Reporting and Social Media Analytics

4.Collaboration Diagrams:

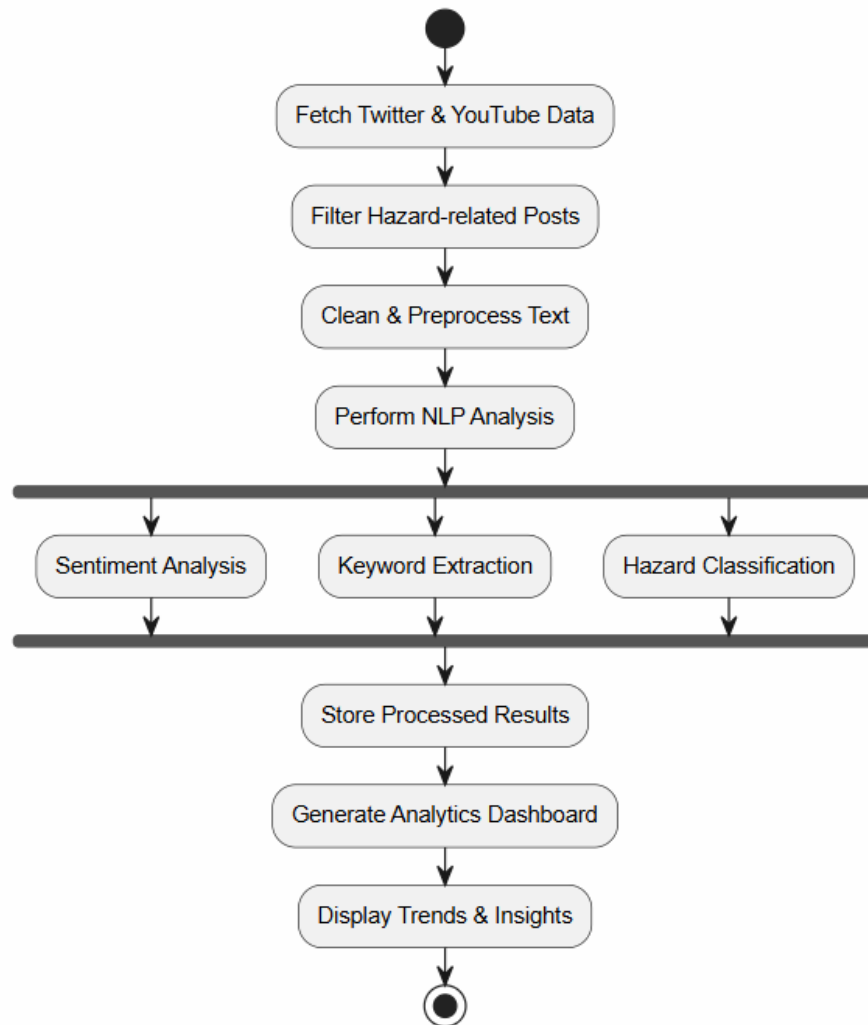


5. Activity Diagram:

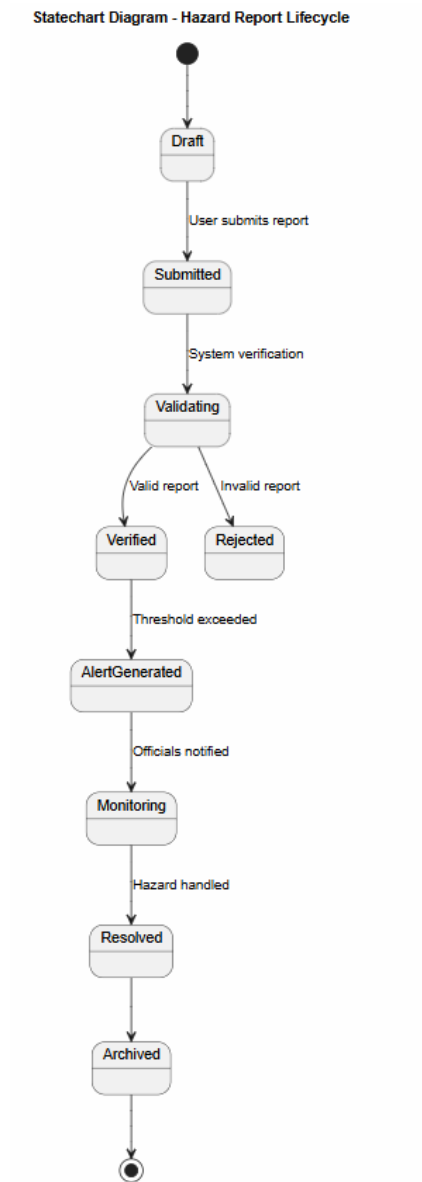
Activity Diagram - Hazard Report Submission



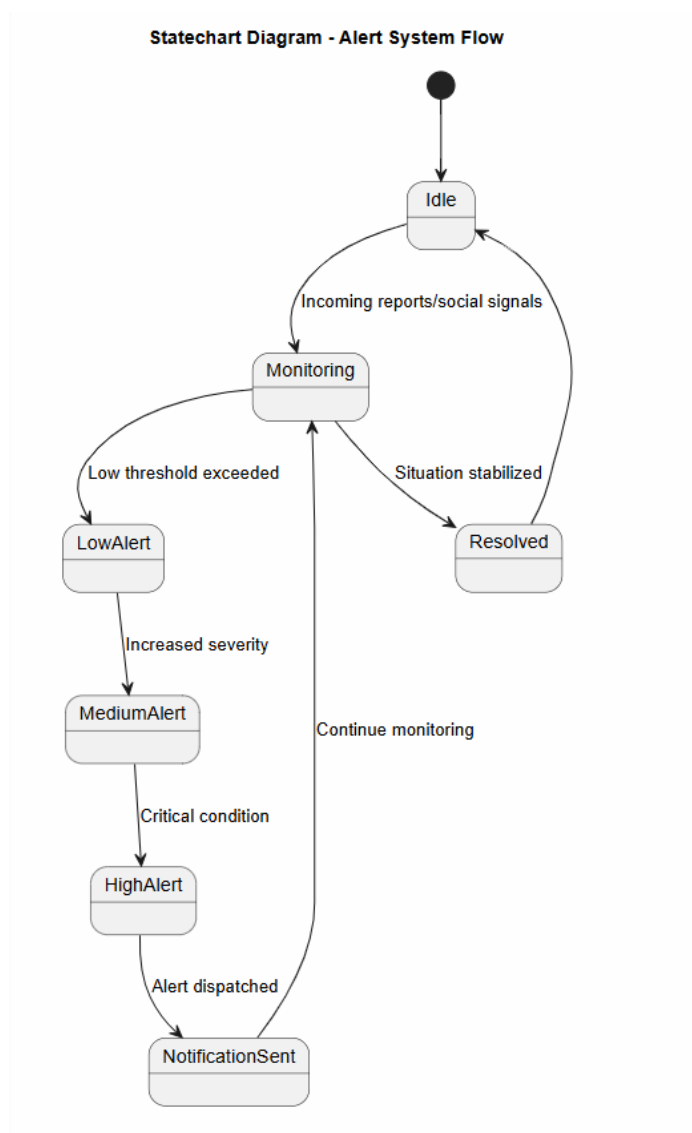
Activity Diagram - Social Media Analytics Workflow



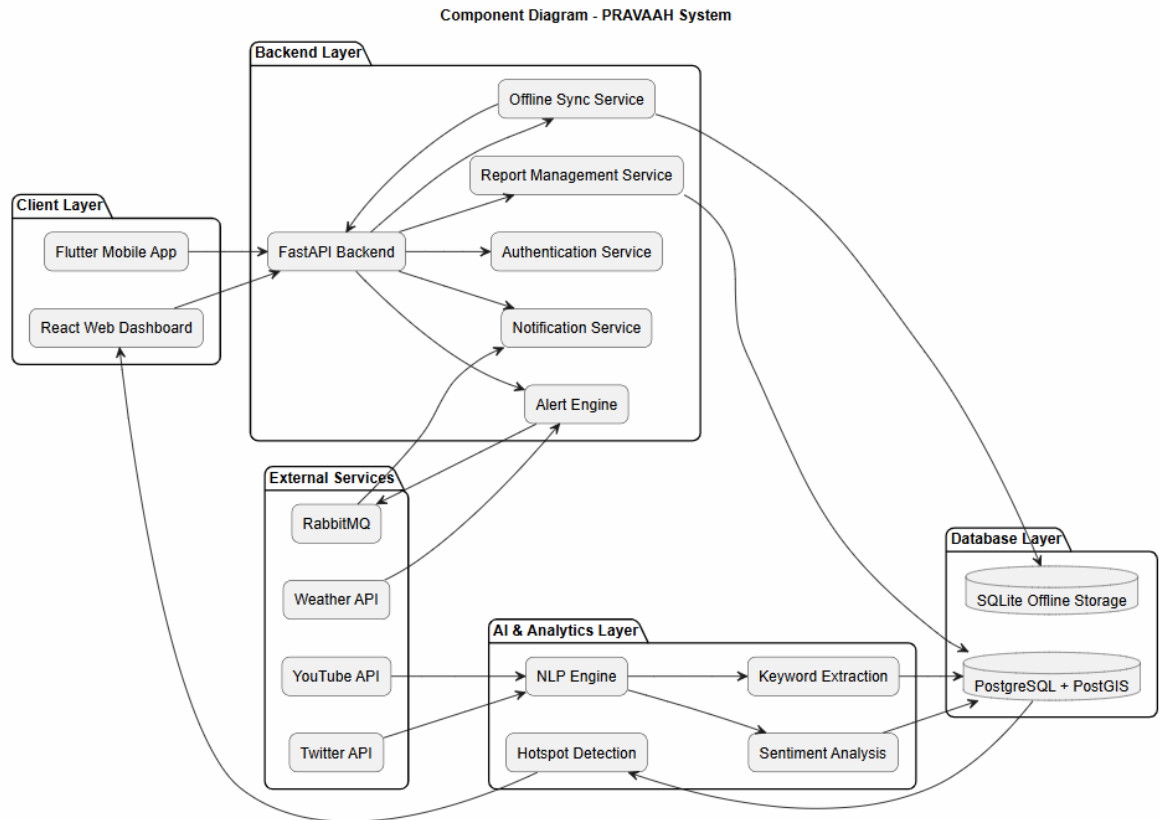
6.StateChart Diagram:



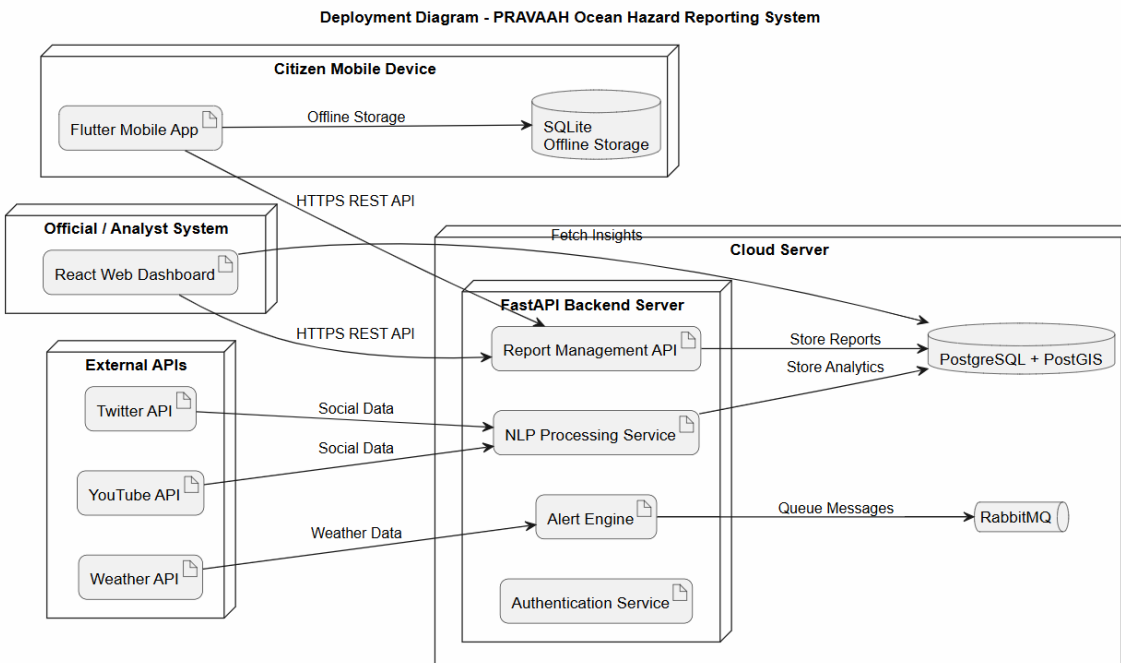
Software Requirements Specification for Integrated Platform for Crowdsourced Ocean Hazard Reporting and Social Media Analytics



7.Component Diagram:



8.Deployment Diagram:



Appendix C: To Be Determined List

The following items are currently identified as TBD and will be finalized in future phases of the project:

1. **Exact threshold values for hazard alert generation** based on report density and severity.
2. **Final selection and tuning of NLP models** for social media analysis.
3. **Integration details with external early warning systems** (e.g., INCOIS APIs).
4. **Multilingual support implementation details** and supported languages.
5. **Cloud deployment configuration** (AWS/Azure/GCP specifics).
6. **Security policy specifications** for data protection and user privacy compliance.
7. **Performance benchmarks under peak load conditions.**